



The Two Pointers Technique

A Comprehensive Handout on Pattern Recognition,
Implementation, and Optimization

Abstract: Two Pointers is an algorithmic technique where two index variables traverse a data structure in a coordinated manner to solve problems in a single pass, converting many brute-force $O(n^2)$ pair-searching problems into $O(n)$ solutions. This handout covers the core patterns—collision (opposite-directional) and sliding window (equi-directional)—along with common pitfalls, clean implementations in C, and practice problems to build mastery.

Contents

S.No	Topic	Page
1	Introduction & Definition	3
2	Real-Life Analogies	4
3	Core Variations	5
	◦ Collision / Opposite-Directional Pointers	5
	◦ Equi-Directional / Sliding Window	6
4	Ambiguities & Common Pitfalls	7
5	Code & Debugging	8
	◦ Container With Most Water	8
	◦ Spot the Bug — Palindrome Checker	9
	◦ Two Sum II	10
	◦ Remove Duplicates	10
6	Pattern Recognition Guide	11
7	Code Examples in C	12
	◦ 3Sum	12
	◦ Subarray with K Different Integers	13
	◦ Longest Substring Without Repeating Characters	14
	◦ Move Zeroes	15
8	Competitive Programming Tips	16
9	Practice Problems — Level Up!	17
10	Common Mistakes	18
11	Time Complexity Cheat Sheet	18
12	Quick Reference	19
13	The Big Picture	21

1 Introduction & Definition

Two Pointers is an algorithmic technique where two index variables (pointers) traverse a data structure — typically an array, string, or linked list — in a coordinated manner to solve a problem in a single pass, or with significantly reduced overhead compared to nested iteration.

Instead of checking *every pair* of elements (i, j) with two nested loops, we exploit some structural property of the data (most commonly **sorted order**, or a **monotonic condition**) to move pointers intelligently, discarding large portions of the search space at each step.

Key Takeaway

Two Pointers converts many brute-force $O(n^2)$ pair-searching problems into $O(n)$ solutions by replacing “check everything” with “eliminate what cannot possibly work.”

1.1 Why It’s Faster: The Complexity Argument

A brute-force approach comparing every pair looks like this:

$$\text{Brute Force} = \sum_{i=1}^n \sum_{j=i+1}^n 1 = \binom{n}{2} = O(n^2)$$

Two Pointers replaces the inner loop with a **single shared traversal**. Each pointer moves *at most* n steps total across the entire algorithm’s lifetime (it never backtracks), so the combined work is bounded by:

$$\text{Two Pointers} = O(n) + O(n) = O(n)$$

This is possible because at each comparison, we can **logically prove** that an entire class of remaining pairs is irrelevant — so we never need to visit them.

Approach	Time Complexity	Space Complexity
Brute Force (Nested Loops)	$O(n^2)$	$O(1)$
Two Pointers	$O(n)$ or $O(n \log n)$ (if sorting first)	$O(1)$

2 Real-Life Analogies

Analogy 1 — The Sorted Shoe Rack

Imagine a shoe rack sorted by size, smallest on the left, largest on the right. You need to find two shoes whose sizes add up exactly to 20 (say, a size 8 and a size 12). Instead of comparing every shoe to every other shoe, you grab the *smallest* and *largest* shoes at once. Too small a sum? Move up from the left. Too big? Move down from the right. You never need to recheck shoes you've already ruled out — that's the **collision** pattern.

Analogy 2 — Reading a Book with a Bookmark and a Finger

Imagine skimming a book: your left hand holds a bookmark (slow pointer) marking the last “valid” paragraph, while your right index finger (fast pointer) scans ahead line by line looking for the next relevant paragraph. When it finds one, you move the bookmark forward and copy the finding into place. You scan the whole book once, but do meaningful “compaction” work only when needed — that's the **equi-directional / sliding window** pattern.

3 Core Variations

3.1 Collision / Opposite-Directional Pointers

Two pointers start at **opposite ends** of a sorted structure and move **toward each other**, shrinking the search window with each step. This is the classic pattern for problems like **Two Sum II** (sorted array), **Container With Most Water**, and **Valid Palindrome**.

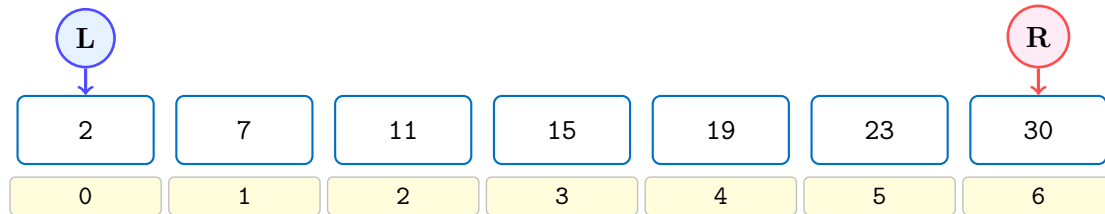


Figure 1: Initial state: L at index 0, R at index 6. Target sum = 26.

Step	Computation	Result	Action
1	$\text{arr}[0] + \text{arr}[6] = 2 + 30 = 32$	Too big	R--
2	$\text{arr}[0] + \text{arr}[5] = 2 + 23 = 25$	Too small	L++
3	$\text{arr}[1] + \text{arr}[5] = 7 + 23 = 30$	Too big	R--
4	$\text{arr}[1] + \text{arr}[4] = 7 + 19 = 26$	FOUND!	Stop

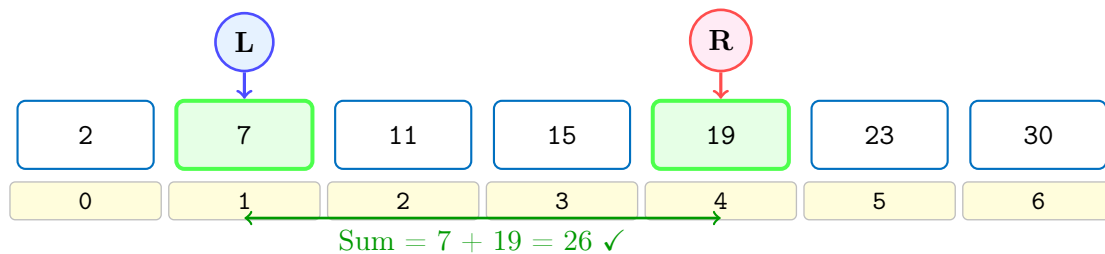


Figure 2: Final state: L=1, R=4. Pair (7, 19) found with sum = 26.

Invariant: Everything strictly between L and R that hasn't been eliminated is still a candidate; anything outside is already proven impossible.

3.2 Equi-Directional / Sliding Window (Fast & Slow)

Both pointers move in the **same direction**, but at potentially different speeds or with a gap between them representing a “window” of elements currently under consideration. Common uses: **removing duplicates in-place**, **cycle detection (Floyd’s algorithm)**, **maximum subarray sum of size k**.

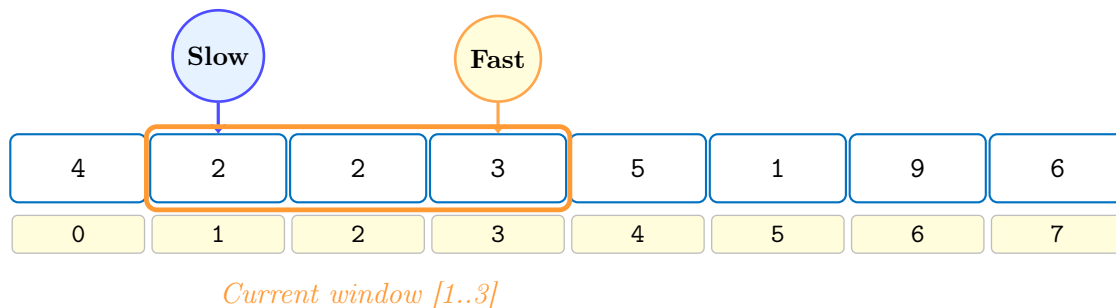


Figure 3: Slow marks the start of the active window; Fast marks the leading edge scanning ahead.

Window progression (size 3, sliding rightward):

Window Indices	Elements	Sum
[0 .. 2]	[4, 2, 2]	8
[1 .. 3]	[2, 2, 3]	7
[2 .. 4]	[2, 3, 5]	10
[3 .. 5]	[3, 5, 1]	9
[4 .. 6]	[5, 1, 9]	15
[5 .. 7]	[1, 9, 6]	16

Key Takeaway

Collision pointers **shrink** a search space; sliding window pointers **expand and contract** a window. Recognizing which one applies is the single most important classification step when reading a new problem.

4 Ambiguities & Common Pitfalls

< vs <= in the Loop Condition

`while (L < R)` stops just before the pointers meet or cross — correct when `L == R` would mean comparing an element to itself (e.g., pair-sum problems).

`while (L <= R)` allows `L == R` — required for problems like binary search, where the single middle element is still a valid candidate.

Using the wrong one is the #1 cause of off-by-one bugs in this pattern.

Forgetting to Skip Duplicates

In problems like **3Sum**, after finding a valid triplet, failing to advance past duplicate values causes the same answer to be recorded multiple times. Always add a `while (arr[L] == arr[L+1]) L++;`-style skip *after* recording a match.

Infinite Loops from Missing Pointer Updates

If **neither branch of your conditional logic** guarantees at least one pointer moves, the loop can spin forever. Always audit every code path to confirm progress is made.

Mutating the Array While Reading It

In in-place compaction (e.g., “Remove Duplicates from Sorted Array”), writing to `arr[slow]` before you’ve *finished reading* `arr[fast]` in the same iteration can silently corrupt data you still need.

Quick Diagnostic Checklist:

- ☐ Is the array/structure required to be **sorted**? If not, do I need to sort first — and can I afford $O(n \log n)$?
- ☐ Does my loop condition correctly handle the case where pointers **meet** vs **cross**?
- ☐ Does **every** branch of my logic move at least one pointer?
- ☐ Am I skipping duplicate values where the problem demands **unique** results?
- ☐ Am I reading a value *before* overwriting its slot, in in-place variants?

5 Code & Debugging

5.1 Clean Implementation — Container With Most Water

Problem: Given n non-negative integers representing heights of vertical lines, find two lines that, together with the x-axis, form a container holding the most water.

```
1  int maxArea(int height[], int n) {
2      /*
3       * Two-pointer solution to 'Container With Most Water'.
4       *
5       * Core insight: the water level is capped by the SHORTER
6       * of the two lines, and the width shrinks every time we
7       * move a pointer inward. So we always move the pointer
8       * at the SHORTER line -- moving the taller one can only
9       * ever decrease or maintain the width while the height
10      * cap stays the same or gets worse.
11      */
12      int left = 0, right = n - 1;
13      int best = 0; // Track the best area seen so far
14
15      while (left < right) {
16          int width = right - left;
17          // The container's height is bounded by the shorter wall
18          int boundedHeight = (height[left] < height[right])
19                               ? height[left]
20                               : height[right];
21          int currentArea = width * boundedHeight;
22          if (currentArea > best)
23              best = currentArea;
24
25          // Move the pointer at the SHORTER line inward.
26          if (height[left] < height[right])
27              left++;
28          else
29              right--;
30      }
31
32      return best;
33  }
```

Complexity: Time $O(n)$ — each pointer moves at most n times total; Space $O(1)$ — only two index variables are used.

5.2 Spot the Bug — Palindrome Checker

The following code is intended to check whether a string is a palindrome. **It contains a bug that causes an infinite loop for any non-empty, non-palindromic string longer than length 1.** Study it before scrolling to the fix.

```
1 int isPalindromeBuggy(char s[]) {
2     int n = strlen(s);
3     int left = 0;
4     int right = n - 1;
5
6     while (left < right) {
7         if (s[left] != s[right])
8             return 0;
9         // ??? something is missing here ???
10    }
11
12    return 1;
13 }
```

▷ Spot the Bug — Take a moment before reading on.

Trace through `s = "abcba"` by hand. What happens to `left` and `right` on each iteration when `s[left] == s[right]`?

Diagnosis and Fix

The Bug: When `s[left] == s[right]`, the loop body does **nothing to advance either pointer**. `left` and `right` never change, so `while (left < right)` re-evaluates the exact same comparison forever — an infinite loop.

The Fix: Increment `left` and decrement `right` on the success path too.

```
int isPalindromeFixed(char s[]) {
    int n = strlen(s);
    int left = 0, right = n - 1;

    while (left < right) {
        if (s[left] != s[right])
            return 0;
        left++;    // Advance inward
        right--;   // Advance inward
    }

    return 1;
}
```

5.3 Two Sum II — Sorted Array

```
1 int twoSum(int arr[], int n, int target) {
2     int left = 0, right = n - 1;
3
4     while (left < right) {
5         int sum = arr[left] + arr[right];
6         if (sum == target) return 1;
7         else if (sum < target)
8             left++;    // Need a larger sum
9         else
10            right--;   // Need a smaller sum
11    }
12
13    return 0;    // No pair found
14 }
```

5.4 Remove Duplicates from Sorted Array (In-Place)

```
1 int removeDuplicates(int arr[], int n) {
2     if (n == 0) return 0;
3
4     int slow = 1;    // Position to place next unique element
5     for (int fast = 1; fast < n; fast++) {
6         if (arr[fast] != arr[fast - 1]) {
7             arr[slow] = arr[fast];
8             slow++;
9         }
10    }
11    return slow;    // New length
12 }
```

6 Pattern Recognition Guide

When you see a new problem, ask these questions in order:

1. **Does the problem involve pairs, subarrays, or substrings?**
 - Pairs $\rightarrow$ collision pointers (usually)
 - Subarrays/substrings $\rightarrow$ sliding window (usually)
2. **Is the data sorted or can it be sorted?**
 - If yes $\rightarrow$ collision pointers are viable
 - If no and sorting would break the problem $\rightarrow$ sliding window or hash map
3. **Are we looking for a contiguous region?**
 - Yes $\rightarrow$ sliding window
 - No $\rightarrow$ collision or other approach

Problem Type	Pattern	Condition
Two Sum (sorted)	Collision	$O(n)$ time, $O(1)$ space
Valid Palindrome	Collision	$O(n)$ time, $O(1)$ space
3Sum	Outer loop + Collision	$O(n^2)$ time, $O(1)$ space
Container With Most Water	Collision	$O(n)$ time, $O(1)$ space
Subarray with K Different Integers	Sliding Window + Freq Map	$O(n)$ time, $O(k)$ space
Max Sum Subarray (size k)	Sliding Window	$O(n)$ time, $O(1)$ space
Longest Substring w/o Repeating	Sliding Window (hash set)	$O(n)$ time, $O(k)$ space
Remove Duplicates (sorted)	Sliding Window (fast/slow)	$O(n)$ time, $O(1)$ space

7 Code Examples in C

7.1 3Sum — Find All Unique Triplets Summing to Zero

```

1  #include <stdlib.h>
2
3  int cmp(const void *a, const void *b) {
4      /*
5       * SAFE three-way comparison.
6       * Do NOT use (*(int*)a - *(int*)b): if a = INT_MAX and b = INT_MIN
7       * the subtraction wraps around (undefined behaviour in C), causing
8       * qsort to produce an incorrect ordering.
9       */
10     int x = *(int*)a, y = *(int*)b;
11     if (x < y) return -1;
12     if (x > y) return 1;
13     return 0;
14 }
15
16 void threeSum(int arr[], int n) {
17     qsort(arr, n, sizeof(int), cmp);
18
19     for (int i = 0; i < n - 2; i++) {
20         // Skip duplicate values for i
21         if (i > 0 && arr[i] == arr[i - 1]) continue;
22
23         int left = i + 1, right = n - 1;
24         while (left < right) {
25             int sum = arr[i] + arr[left] + arr[right];
26             if (sum == 0) {
27                 printf("%d %d %d\n", arr[i], arr[left], arr[right]);
28                 // Skip duplicates for left and right
29                 while (left < right && arr[left] == arr[left + 1]) left++;
30                 while (left < right && arr[right] == arr[right - 1]) right--;
31                 left++;
32                 right--;
33             } else if (sum < 0) {
34                 left++;
35             } else {
36                 right--;
37             }
38         }
39     }
40 }

```

Complexity: Time $O(n^2)$ — sorting is $O(n \log n)$, then the outer loop runs $O(n)$ times with an inner collision-pointer pass; Space $O(1)$ (excluding sorting).

7.2 Subarray with K Different Integers

```

1  #include <string.h>
2
3  /*
4   * CONSTRAINT: arr[i] must satisfy 0 <= arr[i] < MAX_VAL.
5   * On LeetCode the constraint is 1 <= nums[i] <= n, so n <= 20000
6   * satisfies this. For a generic array you must adjust MAX_VAL or
7   * use a hash-map instead of a fixed-size frequency array.
8   */
9  #define MAX_VAL 20001
10
11 // Helper: count subarrays with AT MOST K distinct integers
12 int atMostK(int arr[], int n, int k) {
13     int freq[MAX_VAL] = {0};    /* WARNING: valid only when 0 <= arr[i] <
14     MAX_VAL */
15     int left = 0, count = 0, distinct = 0;
16
17     for (int right = 0; right < n; right++) {
18         if (freq[arr[right]] == 0) distinct++;
19         freq[arr[right]]++;
20
21         while (distinct > k) {
22             freq[arr[left]]--;
23             if (freq[arr[left]] == 0) distinct--;
24             left++;
25         }
26
27         count += right - left + 1;
28     }
29
30     return count;
31 }
32
33 // Exact K distinct = atMost(K) - atMost(K-1)
34 int subarraysWithKDistinct(int arr[], int n, int k) {
35     return atMostK(arr, n, k) - atMostK(arr, n, k - 1);
36 }

```

Complexity: Time $O(n)$, Space $O(\text{MAX_VAL})$ (fixed frequency array under the constraint $0 \leq \text{arr}[i] < \text{MAX_VAL}$). The “at most” trick is the key insight: a sliding window naturally handles “at most” constraints because we can freely shrink the window when the constraint is violated. Converting “exactly K ” into “at most K minus at most $K - 1$ ” lets us reuse the same simple window logic twice.

△ **Constraint Warning:** The freq array above assumes $0 \leq \text{arr}[i] < 20001$. If elements can exceed this bound, increase MAX_VAL or replace the fixed array with a dynamic hash map (e.g., a sorted array + binary search, or a library hash table in C++).

7.3 Longest Substring Without Repeating Characters

```
1 #include <string.h>
2
3 int lengthOfLongestSubstring(char s[]) {
4     int n = strlen(s);
5     int lastIndex[256]; // For ASCII characters
6     for (int i = 0; i < 256; i++) lastIndex[i] = -1;
7
8     int maxLen = 0, start = 0;
9
10    for (int end = 0; end < n; end++) {
11        char c = s[end];
12        // If character was seen within current window, move start
13        if (lastIndex[(int)c] >= start)
14            start = lastIndex[(int)c] + 1;
15
16        // Update last seen position
17        lastIndex[(int)c] = end;
18
19        // Update max length
20        int curLen = end - start + 1;
21        if (curLen > maxLen) maxLen = curLen;
22    }
23
24    return maxLen;
25 }
```

7.4 Move Zeroes (In-Place)

```
1 void moveZeroes(int arr[], int n) {  
2     int pos = 0; // Position to place next non-zero  
3  
4     // Move all non-zero elements to the front  
5     for (int i = 0; i < n; i++) {  
6         if (arr[i] != 0) {  
7             arr[pos] = arr[i];  
8             pos++;  
9         }  
10    }  
11  
12    // Fill remaining positions with zeroes  
13    while (pos < n) {  
14        arr[pos] = 0;  
15        pos++;  
16    }  
17 }
```

8 Competitive Programming Tips

Tip 1: Use scanf/printf for Speed

In competitive programming, fast I/O is critical. Use C's scanf/printf for all input and output.

```
scanf("%d", &n);
int arr[100000];
for (int i = 0; i < n; i++)
    scanf("%d", &arr[i]);
// Solve using two pointers
printf("%d\n", result);
```

Tip 2: Sort First When Needed

Many two-pointer problems require a sorted array. Use `qsort` from `stdlib.h`:

```
int cmp(const void *a, const void *b) {
    int x = *(int*)a, y = *(int*)b;
    if (x < y) return -1;
    if (x > y) return 1;
    return 0;
}
qsort(arr, n, sizeof(int), cmp);
```

Tip 3: Always Check Array Bounds

When advancing pointers, ensure you don't go out of bounds:

```
while (left < right && arr[left] == arr[left + 1])
    left++;
while (left < right && arr[right] == arr[right - 1])
    right--;
```

The `left < right` check prevents accessing invalid indices.

Tip 4: Think in Terms of Invariants

Before coding, write down what is true at each step:

- Collision: "All pairs to the left of L and right of R have been ruled out."
- Sliding window: "The window [slow, fast] satisfies the condition."

This makes correctness proofs and debugging much easier.

9 Practice Problems — Level Up!

Attempt these in order of increasing difficulty. Hints are intentionally partial — the productive struggle is where the pattern truly sinks in.

9.1 [Easy] — Valid Palindrome

Given a string containing letters, digits, and punctuation, determine if it's a palindrome after ignoring non-alphanumeric characters and case.

Hint: Use collision pointers, but add an inner `while` loop on each side to *skip* invalid characters before comparing.

9.2 [Medium] — 3Sum

Given an array of integers, find all unique triplets that sum to zero.

Hint: Sort the array first. Fix one element with an outer loop, then run collision pointers on the remaining subarray for the other two. Don't forget the duplicate-skipping pitfall from Section 4.

9.3 [Medium] — Container With Most Water

Given n non-negative integers representing heights, find the maximum area of water a container can hold.

Hint: Always move the pointer at the shorter line. The width decreases each step, so the only way to increase area is to find a taller boundary.

9.4 [Hard] — Subarray with K Different Integers

Given an array of integers, find the total number of contiguous subarrays containing exactly K unique values.

Hint: This is an advanced sliding window problem. Compute the count of subarrays with *at most* K unique values using a sliding window with a frequency map, then subtract the count of subarrays with *at most* $K - 1$ unique values. This “at most” trick transforms the exact constraint into something a sliding window can handle naturally.

10 Common Mistakes

1. **Wrong loop condition:** Using $\leq$ when you need $<$ (or vice versa).
2. **Not sorting first:** Attempting collision pointers on an unsorted array when the problem requires sorted data.
3. **Forgetting duplicate skips:** In 3Sum and similar problems, missing duplicate handling leads to duplicate output.
4. **Infinite loop:** Missing pointer advancement in one code path.
5. **Off-by-one in window size:** For sliding window of size k , the initial window covers indices 0 to $k-1$.
6. **Writing before reading:** In in-place compaction, overwriting `arr[slow]` before finishing `arr[fast]`.

11 Time Complexity Cheat Sheet

Problem	Time	Space
Two Sum (sorted)	$O(n)$	$O(1)$
Valid Palindrome	$O(n)$	$O(1)$
3Sum	$O(n^2)$	$O(1)$ or $O(n)$
Container With Most Water	$O(n)$	$O(1)$
Subarray with K Different Integers	$O(n)$	$O(k)$
Max Sum Subarray (size k)	$O(n)$	$O(1)$
Longest Substring (no repeat)	$O(n)$	$O(k)$
Move Zeroes	$O(n)$	$O(1)$

12 Quick Reference

12.1 Collision Pointers Template

```
int twoPointers(int arr[], int n, int target) {
    int left = 0, right = n - 1;

    while (left < right) {
        int current = /* compute from arr[left] and arr[right] */;

        if (current == target)
            return /* result */;
        else if (/* condition to move left */)
            left++;
        else
            right--;
    }

    return /* default */;
}
```

12.2 Sliding Window Template

```
int slidingWindow(int arr[], int n, int k) {
    // Build initial window
    int window = 0;
    for (int i = 0; i < k; i++)
        window += arr[i];

    int best = window;

    // Slide the window
    for (int i = k; i < n; i++) {
        window += arr[i] - arr[i - k];
        if (window > best) best = window;
    }

    return best;
}
```

12.3 Fast/Slow Pointer Template (In-Place)

```
int fastSlowPointers(int arr[], int n) {
    int slow = 0;
    for (int fast = 0; fast < n; fast++) {
        if (/* condition for keeping arr[fast] */) {
            arr[slow] = arr[fast];
            slow++;
        }
    }
}
```

```
    }  
    return slow; // New length  
}
```

13 The Big Picture

Here's what we've learned:

1. Two Pointers converts $O(n^2)$ brute-force pair searching to $O(n)$.
2. **Collision pointers** (opposite ends, moving inward) are for sorted data, pair-finding, and palindrome problems.
3. **Sliding window** (same direction, maintaining a window) is for subarrays, substrings, and in-place compaction.
4. Every branch must advance at least one pointer — infinite loops are the most common bug.
5. Duplicate skipping, correct loop conditions, and reading-before-writing are critical details.

Final Key Takeaway

Before writing a single line of code, classify the problem: **Collision** (shrinking search space, usually on sorted data) or **Sliding Window** (expanding/contracting a contiguous region). The pattern you choose dictates your loop condition, your pointer movement rules, and where your bugs are most likely to hide.

— Keep Practicing! —